

RESEARCH ARTICLE

Interaction-Aware Space–Time Homotopy Constraints for Path–Velocity Planning in Dynamic Multi-Obstacle Traffic

JiaWang Liao^a, YuFei Hu^a, ChaoYang Shi^a and ChengJiao Sun^a

^aSchool of Computer Engineering, Hubei University of Arts and Science, 296 Longzhong Road, Xiangyang 441053, Hubei, China

ARTICLE HISTORY

Manuscript prepared for peer review

ABSTRACT

Path–velocity decomposition supports efficient planning but can discard interaction timing: time-blind path bounds merge occupancies that occur at different times, while obstacle-wise side decisions can create false blockage. We develop MIKU, an interaction-aware constraint interface that retains the downstream path and speed quadratic programs. Arrival-time-dependent projection prevents temporally disjoint occupancies from being merged; threat-aware margins and a continuous partition generate group-level lateral bands; and a temporal homotopy graph compiles safe passage windows into station–time box constraints. For the fixed-section interval model, the maximum-gap scan reduces the k -obstacle side-assignment search from $O(2^k)$ to $O(k \log k)$ and matched exhaustive enumeration on 4,000 deterministic interval instances. Across 3,500 paired simulated scenarios covering seven traffic configurations, MIKU increased collision-free goal reaching from 62.7% to 77.5% (difference 14.8 percentage points, 95% CI 13.5–16.0) and reduced collision from 2.2% to 1.1%, while mean progress increased by 8.9 percentage points. Median and 95th-percentile planning times were 9.98 and 55.93 ms, although the 99th percentile was 221.8 ms versus 138.9 ms for the baseline. A separate 700-scenario rolling-replanning study and Apollo 11.0 interface cases support replanning behavior and software integration, but do not constitute road testing. The results show how preserving interaction timing at the constraint interface can reduce unnecessary stopping without replacing established convex optimizers, subject to tail-latency and simulation-to-road validation.

KEYWORDS

autonomous driving; path–velocity decomposition; space–time homotopy; dynamic obstacles; motion planning

1. Introduction

Automated vehicles must make progress through dynamic traffic while maintaining collision avoidance and ride quality. Excessively conservative motion planning is therefore not benign: repeated fallback stops can reduce service reliability, obstruct surrounding traffic, and erode the mobility benefit expected from an intelligent transportation system. Motion-planning surveys consequently treat feasibility, safety, comfort, and computation as coupled operational outcomes rather than interchangeable measures (González, Pérez, Milanés, & Nashashibi, 2016; Paden, Čáp, Yong, Yershov, & Frazzoli, 2016; Schwarting, Alonso-Mora, & Rus, 2018).

Path–velocity decomposition (PVD) is attractive for online operation because it separates lateral path generation in station–lateral (SL) coordinates from longitudinal speed optimization in station–time (ST) coordinates (Kant & Zucker, 1986; Werling, Ziegler, Kammel, & Thrun, 2010; Zhou et al., 2021). Industrial implementations can then solve two structured quadratic programs (QPs) and expose environmental information through path and ST boundaries (Fan et al., 2018; Wang et al., 2022; Y. Zhang et al., 2020). The separation, however, can discard precisely the information needed in dense dynamic scenes. A path boundary formed from a current snapshot

limitations, and conclusions.

2. Related work

2.1. Joint space–time planning

Joint planners retain geometric and temporal coupling by searching or optimizing an expanded state. Conformal spatiotemporal lattices coordinate position, velocity, and time directly (McNaughton et al., 2011); instantaneous-analysis and SLT formulations similarly combine lateral and longitudinal decisions in dense traffic (Hu et al., 2023, 2025; Qiao, 2025; X. Yang & Li, 2021). Mixed-integer formulations can encode obstacle-side choices as discrete variables (Kessler et al., 2023). These approaches provide strong coordination, but their expanded search spaces, integer decisions, or nonlinear programs differ structurally from the two sequential convex QPs used by many PVD stacks. The finite-grid joint search used later in this paper is therefore a computational reference rather than a claimed continuous optimum.

2.2. Decomposed and iterative coordination

PVD planners obtain tractability by optimizing a path first and then a speed profile (Kant & Zucker, 1986; Werling et al., 2010). Convex smoothing and piecewise-jerk optimization make this structure suitable for production-oriented software (Fan et al., 2018; Y. Zhang et al., 2020; Zhou et al., 2021). Temporal optimization, iterative anchoring, candidate pairing, and risk-aware coordination can return information between the two stages (Chen, Zhan, & Tomizuka, 2019; Han, Leng, Zhang, & Xiong, 2026; C. Liu, Zhan, & Tomizuka, 2017; F. Liu, Zhao, Su, & Wen, 2024; Z. Zhang et al., 2023). Their usefulness depends on what information each stage exposes: repeated arrival-time updates cannot recover a lateral alternative that was discarded by an irreversible local side decision.

Safe-corridor methods make the feasible region explicit before continuous optimization. Spatial flight corridors demonstrate how convex sets can bridge discrete search and trajectory optimization (S. Liu et al., 2017), while recent autonomous-driving work extends this idea to spatiotemporal lane-change or Frenet corridors (Tariq, Yeh, Singh, Isele, & Bae, 2025; Yoon et al., 2024). MIKU shares the explicit-feasible-region principle but targets a narrower interface question: how to generate mutually consistent SL and ST bounds without replacing the path and speed QPs.

2.3. Interaction prediction and safety margins

Prediction-aware planning can account for pedestrian and vehicle interactions that a static obstacle footprint misses (Jeong & Yi, 2021; B. Yang, Yan, Wang, & Nakano, 2023). Such predictions remain uncertain, so a planner must distinguish a nominal arrival estimate from the occupancy set used for collision checking. MIKU uses the former to query and rank alternatives, while bounded position and velocity errors expand the latter. Threat-dependent extra margins further allocate scarce lateral space according to collision urgency, overlap, relative motion, object type, and local density; time-to-collision is used only as one normalized urgency factor (Minderhoud & Bovy, 2001).

Table 1 summarizes the solver-level distinction. Existing families are effective under their stated assumptions; the unresolved case is a PVD stack that must preserve its downstream QPs while jointly coordinating time-dependent projection, group-level side selection, and speed-stage timing constraints.

Table 1. Relationship between planning families and the constraint-interface problem.

Family	Space-time coordination	Solver implication	Interface-level gap addressed here
Time-blind PVD	Sequential path then speed	Two structured QPs	Loses arrival-conditioned occupancy and group choices
Iterative PVD	Repeated path-speed updates	Multiple solver calls	Cannot restore discarded lateral classes by timing alone
Joint lattice or mixed-integer planning	Coupled state or discrete variables	Expanded search or a different optimizer	Does not preserve the original two-QP interface
Safe-corridor planning	Convex feasible sets before optimization	Corridor generator plus continuous solver	Often focuses on a selected corridor rather than coupled spatial and temporal classes
MIKU	Spatial and temporal homotopies compiled into bounds	Retains the two downstream QPs	Targets arrival timing, group decisions, and cross-stage constraint transfer

3. Problem formulation and evidence scope

3.1. Path-velocity interface

We consider a road-aligned Frenet frame in which s denotes station and l denotes lateral displacement from a reference line. A PVD planner first optimizes a lateral path $l(s)$ in SL space and then optimizes a longitudinal motion $s(t)$ in ST space. With path states $\mathbf{x}_p = (l_j, l'_j, l''_j)_{j=0}^K$ and speed states $\mathbf{x}_v = (s_r, v_r, a_r)_{r=0}^N$, the common downstream solver can be written as

$$\mathbf{x}_p^* = \arg \min_{\mathbf{x}_p} \frac{1}{2} \mathbf{x}_p^\top H_p \mathbf{x}_p + q_p^\top \mathbf{x}_p, \quad (1)$$

$$\text{s.t. } A_p \mathbf{x}_p = b_p, \quad l_j^- \leq l_j \leq l_j^+, \quad \mathbf{x}_v^* = \arg \min_{\mathbf{x}_v} \frac{1}{2} \mathbf{x}_v^\top H_v \mathbf{x}_v + q_v^\top \mathbf{x}_v, \quad (2)$$

$$\text{s.t. } A_v \mathbf{x}_v = b_v, \quad s_r^{lb} \leq s_r \leq s_r^{ub}, \quad 0 \leq v_r \leq v_{\max}, \quad a_{\min} \leq a_r \leq a_{\max}. \quad (3)$$

Here H_p and H_v encode smoothness costs, while A_p and A_v encode the piecewise-jerk kinematic equalities. The path-boundary set $\mathcal{B}_l = \{[l_j^-, l_j^+]\}$ and the ST position-boundary set $\mathcal{B}_s = \{[s_r^{lb}, s_r^{ub}]\}$ are the environmental interfaces. MIKU changes these interfaces but leaves the downstream objective matrices, kinematic equalities, and QP solver unchanged.

3.2. Center-coordinate obstacle model

For obstacle i , let $O_i(t) = [s_i^-(t), s_i^+(t)] \times [l_i^-(t), l_i^+(t)]$ be its physical rectangle. If the physical road limits are $[l_{\text{road}}^-, l_{\text{road}}^+]$, ego width is W_{ego} , and the roadside margin is d_{road} , the feasible interval for the ego center is

$$[L^-, L^+] = [l_{\text{road}}^- + W_{\text{ego}}/2 + d_{\text{road}}, l_{\text{road}}^+ - W_{\text{ego}}/2 - d_{\text{road}}]. \quad (4)$$

The obstacle interval passed to the center-coordinate planner is expanded once by half the ego width and the obstacle-specific extra margin $d_{\text{buf},i}$:

$$[u_i, v_i] = [l_i^- - W_{\text{ego}}/2 - d_{\text{buf},i}, l_i^+ + W_{\text{ego}}/2 + d_{\text{buf},i}]. \quad (5)$$

All four quantities in (4)–(5) refer to the ego center. Thus a gap width is a center-placement width; vehicle width is not subtracted again in later feasibility tests.

For an active set of k obstacles at one station, assign each obstacle to the left (L) or right

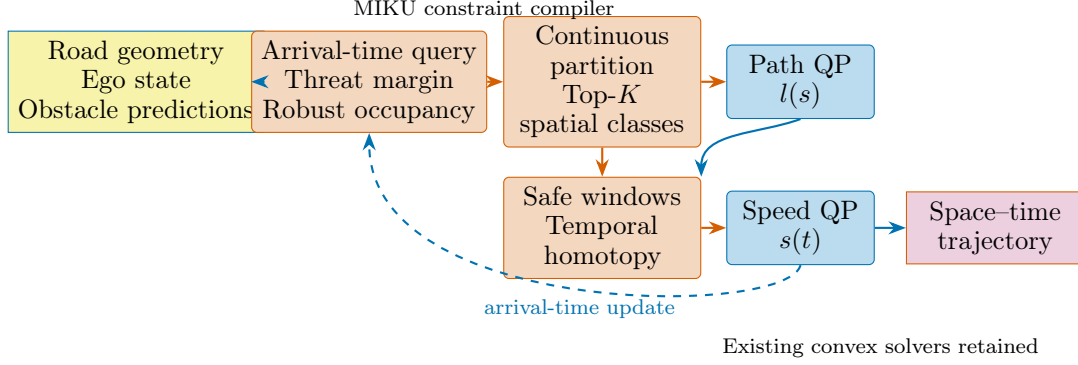


Figure 2. MIKU constraint construction and transfer through a PVD planning stack.

(R) side of the ego band. For decision vector $\mathbf{d}$, the resulting center band is

$$l^-(\mathbf{d}) = \max \left(L^-, \max_{i \in \mathcal{L}} v_i \right), \quad l^+(\mathbf{d}) = \min \left(L^+, \min_{i \in \mathcal{R}} u_i \right), \quad (6)$$

with empty extrema omitted. The band is feasible when $W(\mathbf{d}) = l^+(\mathbf{d}) - l^-(\mathbf{d}) \geq \varepsilon$ for a numerical tolerance ε .

3.3. Two failure mechanisms and study boundary

When the path stage ignores time, it may replace a dynamic occupancy by the horizon union

$$\overline{O}_i = \bigcup_{t \in [0, T_{\text{pred}}]} O_i(t), \quad (7)$$

which treats positions occupied at different times as simultaneous. Conversely, an obstacle-wise greedy policy may commit to a side before seeing the effect of another obstacle in the same longitudinal component. Either mechanism can produce $l^+(s) < l^-(s)$ even though a temporally consistent passage exists.

The study tests whether reconstructing the missing arrival-time and group-level information at the boundary interface improves planning outcomes. The principal evidence is generated on straight-road scenarios with constant-velocity obstacle models and bounded perturbations. The 3,500-case experiment is a matched numerical evaluation, not a public-data or field study. Apollo Planning is used only for interface and representative behavior evidence; it is not treated as a physical-road test or a native runtime benchmark.

4. MIKU methodology

4.1. Overview

MIKU reconstructs the two pieces of information lost by time-blind and obstacle-wise PVD. It first estimates the ego arrival time $\tau(s)$ and queries time-dependent obstacle occupancy. It then computes threat-dependent center bounds, groups longitudinally connected obstacles, and selects continuous lateral bands. After the path QP, uncertainty-expanded occupancy tubes define safe passage windows. A layered temporal homotopy graph selects a causally compatible window sequence and writes it into the ST position bounds. Figure 2 summarizes the data flow.

4.2. Threat-aware margins

For obstacle i , MIKU combines collision urgency, lateral overlap, relative motion, object type, and local interaction density into a normalized threat score:

$$\Theta_i = w_1 f_{\text{TTC}}(i) + w_2 f_{\text{overlap}}(i) + w_3 f_{\text{vel}}(i) + w_4 f_{\text{type}}(i) + w_5 f_{\text{inter}}(i). \quad (8)$$

All factors lie in $[0, 1]$, and the weights are $(0.30, 0.20, 0.15, 0.10, 0.25)$. When the ego vehicle is behind an obstacle and closing, $\text{TTC}_i = (s_i - s_{\text{ego}})/(\dot{s}_{\text{ego}} - \dot{s}_i)$; otherwise it is $+\infty$. The piecewise-linear TTC factor equals one for $\text{TTC}_i \leq 2.0$ s, zero for $\text{TTC}_i \geq 7.0$ s, and decreases linearly in between (Minderhoud & Bovy, 2001). The overlap and relative-speed factors are

$$f_{\text{overlap}}(i) = \min\left(1, \frac{\max(0, \min(l_i^+, l_{\text{ego}}^+) - \max(l_i^-, l_{\text{ego}}^-))}{\max(0.1, \min(W_i, W_{\text{ego}}))}\right), \quad (9)$$

$$f_{\text{vel}}(i) = \sigma(v_{\text{rel},i}/v_{\text{max}}), \quad \sigma(x) = \frac{1}{1 + e^{-5x}}. \quad (10)$$

Here $W_i = l_i^+ - l_i^-$ is the obstacle width; the cap keeps the overlap factor in $[0, 1]$ for obstacles wider than the ego vehicle. The type factors for vulnerable road users, motor vehicles, unknown movable objects, static obstacles, and small markers are respectively 1.0, 0.7, 0.5, 0.3, and 0.15. Density uses a 10 m neighborhood and is zero for a single obstacle. The extra margin is

$$d_{\text{buf},i} = d_{\text{buf},\min} + (d_{\text{buf},\max} - d_{\text{buf},\min})\Theta_i, \quad (11)$$

where $[d_{\text{buf},\min}, d_{\text{buf},\max}] = [0.10, 0.40]$ m. This term changes the boundary value only; it does not change the sorting or scan structure.

4.3. Continuous lateral partition and maximum gap

Obstacles whose longitudinal intervals overlap are assigned to the same connected component. Sorting by the lower station edge and maintaining the largest upper edge constructs all components in $O(n \log n)$. Within one component, sort the active intervals by u_i (ties by descending v_i). The following lemma limits the side-assignment search.

Lemma 4.1 (Continuous optimal partition). *For the center-band objective in (6), there is an optimal assignment in which the obstacles sorted by u_i form a prefix assigned to L and a suffix assigned to R . Hence only $k + 1$ split points need be considered.*

Proof. If both sides are nonempty, let $\beta = \min_{i \in \mathcal{R}} u_i$. Move every obstacle currently in $\mathcal{L}$ with $u_i \geq \beta$ to $\mathcal{R}$. The upper bound remains unchanged because each moved lower edge is no smaller than β , whereas the lower bound cannot increase after removing elements from $\mathcal{L}$. The width therefore does not decrease. Repeating this operation yields a continuous split. Empty-side cases give $p = 0$ or $p = k$. Descending v_i resolves ties in u_i . $\square$

For split p , define the prefix maximum and candidate upper edge by

$$V_0 = L^-, \quad V_p = \max\left(L^-, \max_{1 \leq q \leq p} v_{(q)}\right), \quad (12)$$

$$U_p = \begin{cases} \min(L^+, u_{(p+1)}), & p < k, \\ L^+, & p = k, \end{cases} \quad g_p = U_p - V_p. \quad (13)$$

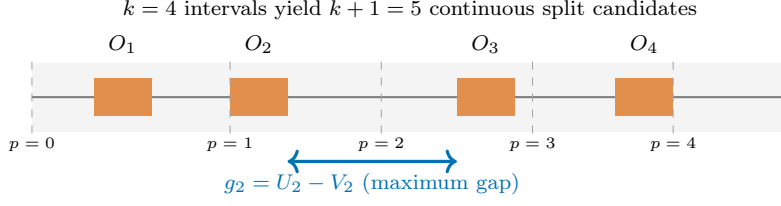


Figure 3. Continuous split candidates for the fixed-section maximum-gap problem. Four sorted forbidden intervals induce five candidate splits; the prefix maximum is required when interval upper edges are nested.

Theorem 4.2 (Maximum-gap optimality). *For the fixed-section interval model, the optimal center-band width is $W^* = \max_{p=0,\dots,k} g_p$. The candidates are obtained by sorting and one prefix scan, with complexity $O(k \log k)$.*

Proof. For split p , (6) gives the lower edge V_p and the tightest upper edge U_p , so its width is g_p . Lemma 4.1 guarantees an optimal continuous split; enumerating the $k + 1$ splits therefore attains the optimum. $\square$

The theorem is exact only for this fixed-section interval abstraction. It does not certify a continuous global optimum for the full nonconvex trajectory problem. MIKU keeps the $Q = 3$ widest positive bands in each component and uses dynamic programming to balance band width against lateral-center changes. If band B_{jq} has center c_{jq} and width g_{jq} in component j , the sequence minimizes

$$-\sum_j g_{jq_j} + \lambda_s \left(|c_{1q_1} - l_0| + \sum_{j>1} |c_{jq_j} - c_{j-1,q_{j-1}}| \right), \quad (14)$$

with the recurrence

$$D_{jq} = -g_{jq} + \min_r \{D_{j-1,r} + \lambda_s |c_{jq} - c_{j-1,r}| \}. \quad (15)$$

4.4. Arrival-time-dependent projection

At station s , the path stage queries obstacle occupancy at the estimated ego arrival time. With the current state and a constant-acceleration approximation,

$$\tau(s) = \frac{-v_{\text{ego}} + \sqrt{v_{\text{ego}}^2 + 2a_{\text{ego}}(s - s_{\text{ego}})}}{a_{\text{ego}}}, \quad (16)$$

with the constant-speed limit used when $|a_{\text{ego}}| < 10^{-3}$. A negative discriminant is marked outside the planning horizon. In rolling replanning, $\tau(s)$ is interpolated from the previous speed solution and updated at each cycle.

The numerical scenarios use $s_i(t) = s_{i,0} + v_{s,i}t$ and $l_i(t) = l_{i,0} + v_{l,i}t$. For bounded prediction errors, the longitudinal and lateral expansion at time t are

$$\epsilon_{s,i}(t) = \epsilon_{s0,i} + t\epsilon_{vs,i}, \quad \epsilon_{l,i}(t) = \epsilon_{l0,i} + t\epsilon_{vl,i}. \quad (17)$$

The physical obstacle and ego footprint are expanded by these quantities before the ST occupancy map is formed. Nominal $\tau(s)$ selects and ranks alternatives; collision checking uses the expanded occupancy.

4.5. Temporal homotopy and ST transfer

For conflict region c , let $I_{cr} = [t_{cr}^{in}, t_{cr}^{out}]$ be each overlap interval between the path and the predicted occupancy tube. After temporal safety expansion η_c , the safe windows are

$$\mathcal{W}_c = [0, T] \setminus \bigcup_r [t_{cr}^{in} - \eta_c, t_{cr}^{out} + \eta_c]. \quad (18)$$

Intersecting these windows with the earliest reachable arrival time yields $W_{cq} = [\alpha_{cq}, \beta_{cq}]$. For conflict points ordered by station, connect adjacent nodes when

$$\hat{t}_c \geq \hat{t}_{c-1} + \frac{s_c - s_{c-1}}{v_{\max}}, \quad \hat{t}_c \in W_{cq}. \quad (19)$$

Node cost measures deviation from $\tau(s_c)$, and edge cost discourages unnecessary switching between before- and after-obstacle passage. A bounded-width dynamic program selects one causal sequence.

For a selected window, its semantics enter the speed QP as linear box constraints:

$$\begin{aligned} s_j^{ub} &= \min(s_j^{ub,0}, s_c^- - \epsilon_s), & t_j &< \alpha_{cq}, \\ s_j^{lb} &= \max(s_j^{lb,0}, s_c^+ + \epsilon_s), & t_j &\geq \beta_{cq}. \end{aligned} \quad (20)$$

If no complete window sequence is reachable, the planner imposes a stopping upper bound before the first unreachable conflict. The downstream speed QP remains a convex QP with the same objective and kinematic equalities.

4.6. Algorithm and complexity

Algorithm 1. MIKU constraint compilation

Input: Road bounds, ego state, and predicted trajectories for n obstacles

Output: Center path bounds and ST position bounds

- 1 Estimate $\tau(s)$ and compute threat-dependent center intervals for each obstacle.
 - 2 Sort station intervals and form longitudinal connected components.
 - 3 **foreach** *component* **do**
 - 4 $\lfloor$ Sort by $(u_i, -v_i)$, compute prefix maxima, and retain the Q widest positive bands.
 - 5 Select a continuous spatial-homotopy sequence by (14).
 - 6 Write the selected side assignments into the active path bounds at each station.
 - 7 Construct uncertainty-expanded occupancy tubes and safe windows.
 - 8 Select a causal temporal-homotopy sequence by (19) and compile it into (20).
 - 9 If enabled, update $\tau(s)$ from the speed solution and repeat until the iteration limit or convergence.
 - 10 **return** *Path and ST bounds*
-

For K path samples, m components, at most Q spatial candidates per component, and a bounded temporal beam width B , sorting and grouping cost $O(n \log n)$. With K_s retained prefixes per terminal spatial band, the layered spatial dynamic program costs $O(mK_s Q^2 \log(K_s Q))$; temporal beam expansion and ranking cost $O(hBR \log(BR))$ for h conflict points and R reachable windows per layer. The implementation uses $Q = 3$, $K_s = 3$, and $B = 8$. Occupancy queries are $O(nK)$ and naive interaction-density computation is $O(n^2)$ in the worst case.

5. Experimental design

The evaluation separates statistical planning evidence from mechanism and software-integration evidence. The statistical experiments use deterministic scenario generators and fixed seeds so that every method receives the same planning input and the same vehicle constraints. Collision and clearance are evaluated against a separately retained truth scenario. Apollo Planning is used in a second evidence stream to check interface compatibility and representative behavior.

5.1. Protocols and scenarios

The main open-loop protocol contains 3,500 paired scenarios, with 500 seeds in each of seven families: crossing pedestrians, vehicle cut-ins, parked vehicles with oncoming traffic, narrow multi-obstacle passages, interleaved dynamic obstacles, prediction noise, and delayed crossings. Obstacle and ego parameters are sampled within the family ranges, while the seed and family determine the complete case. The planning and truth scenarios are identical except in the prediction-noise family, where bounded errors are injected only into the planning input. The generators, raw rows, metadata, and hashes are provided in the supplement.

The 4,000-instance geometry check uses 20 deterministic seeds and 200 fixed-section interval instances per seed. Each instance contains zero to eight possibly overlapping, nested, or road-crossing forbidden intervals. The scan is compared with exhaustive enumeration of all 2^k side assignments. This experiment tests center-band width equality under the stated interval model, not tracking error or full-trajectory optimality.

To test rolling behavior, an independent 700-scenario protocol uses 100 seeds per family. Every 0.5 s, the first planned segment is executed, the ego and obstacles are updated, and the planner is called again until arrival or the scenario horizon. These cases are not pooled with the 3,500 open-loop cases. Finally, a 70-case stratified finite-grid joint reference uses 10 seeds per family and grid spacings $\Delta t = 0.5$ s, $\Delta s = 0.5$ m, $\Delta l = 0.25$ m, and $\Delta v = 1.0$ m/s, with beam width 20,000. It is explicitly a coarse finite search, not a continuous global oracle.

5.2. Baselines and implementation

The matched PVD baselines are defined by the information exchanged at the constraint interface:

- B0 is time-blind and uses the fixed local side policy.
- B1 adds arrival-time-dependent projection but retains obstacle-wise greedy side selection.
- B2 allows three arrival-time refinement iterations in the B1 policy.
- MIKU adds group-level continuous partition, Top- K spatial homotopy, threat-aware margins, uncertainty-expanded occupancy, temporal homotopy, and at most two planning iterations with alternating updates.

All four methods share the same path and speed QP objectives, kinematic constraints, scenario input, and numerical metric implementation. B3 is the finite-grid joint reference and uses a different solver by design. The implementation was evaluated on an Intel i9-14900HX processor with 64 GB RAM and Arch Linux. The Apollo integration uses the Apollo 11.0 path-boundary and ST-boundary interfaces. It confirms that the compiled constraints can enter the existing planning chain; it does not provide a native Apollo benchmark.

5.3. Outcomes and statistical analysis

The primary outcome is collision-free goal reaching: a run is successful when it remains collision-free over the full horizon and reaches $s_{\text{end}} \geq s_{\text{max}} - 1$ m. Collision rate is evaluated against truth rectangles, and degraded-stop rate records runs that stop or fail to reach the goal without a collision. Progress ratio, average speed, jerk RMS, minimum signed clearance, minimum longitudinal TTC, penalized travel time, and complete planning latency provide operational,

Table 2. Results on 3,500 paired scenarios. Values are percentages for rates, means for continuous outcomes, and milliseconds for latency.

Outcome	B0	B1	B2	MIKU
Collision-free goal reaching (%)	62.69	52.37	62.09	77.46
Collision (%)	2.17	2.14	2.23	1.14
Degraded stop (%)	35.40	45.77	35.97	21.51
Progress ratio (%)	77.52	71.07	76.62	86.45
Jerk RMS (m/s^3)	1.252	1.315	1.300	1.132
Runtime P50 (ms)	10.11	11.37	31.20	9.98
Runtime P95 (ms)	56.79	63.24	169.49	55.93
Runtime P99 (ms)	138.88	169.73	463.05	221.80

safety, comfort, and computational context. Penalized travel time equals arrival time for collision-free arrivals and the scenario horizon plus 5 s otherwise. Runtime includes the complete method call; B2 includes all coordination iterations.

For each fixed seed, we compute MIKU minus baseline differences before aggregation. Continuous metrics use 5,000-resample paired percentile bootstrap intervals. Binary success and collision outcomes use exact McNemar tests. Effect sizes are paired Cohen’s d_z . Runtime is reported at the 50th, 95th, and 99th percentiles because a typical-cycle value alone can conceal an operational tail. No external leaderboard comparison is made: B0–B2 are internal matched variants, and B3 is a finite computational reference.

5.4. Mechanism and interface cases

Four deterministic mechanism cases expose the two target failure modes: time-blind occupancy and incompatible obstacle-wise side choices. Four paired control cases alter only the time horizon or lateral placement so that the baseline is feasible. Three Apollo Dreamview cases illustrate dynamic overtaking, a narrow passage, and single-lane diversion. They are qualitative interface cases and are reported separately from the numerical protocols.

6. Results

6.1. RQ1: Main comparison

The fixed-section test found exact agreement between the $O(k \log k)$ scan and exhaustive enumeration for all 4,000 generated instances. The maximum absolute center-band-width difference was below the 10^{-12} numerical test tolerance. This result establishes the theorem’s fixed-section claim only; it does not imply global optimality of the complete trajectory planner.

Table 2 reports the 3,500 paired scenarios. MIKU increased collision-free goal reaching from 62.69% for B0 to 77.46%, a paired difference of 14.77 percentage points (95% CI 13.54–15.97; exact McNemar $p = 1.94 \times 10^{-133}$). Collision decreased from 2.17% to 1.14% (difference -1.03 percentage points, 95% CI -1.37 to -0.71 ; $p = 2.91 \times 10^{-11}$). Mean progress increased by 8.93 percentage points, jerk RMS decreased by 0.120 m/s^3 , and penalized travel time decreased by 0.81 s. The aggregate mean-clearance difference was -0.010 m with a 95% CI of $[-0.024, 0.004]$, so the success and collision gains cannot be interpreted as a uniform clearance increase.

The family-level effects in Figure 4 show where the overall difference originates. The largest success gain occurs in the narrow multi-obstacle family (65.4 percentage points), followed by delayed crossing (26.8 percentage points). Crossing-pedestrian and vehicle cut-in families show no aggregate success difference in this protocol, which is consistent with cases in which the baseline already has a feasible temporal response. Prediction noise yields a 3.2-point success increase and a 7.0-point collision reduction, but progress decreases by 1.12 points and jerk RMS increases by 0.041 m/s^3 .

The finite-grid reference provides a computational trade-off rather than a winner-takes-all comparison. In 70 cases, B3 reached 80.0% of goals and MIKU 77.1% (paired difference -2.86

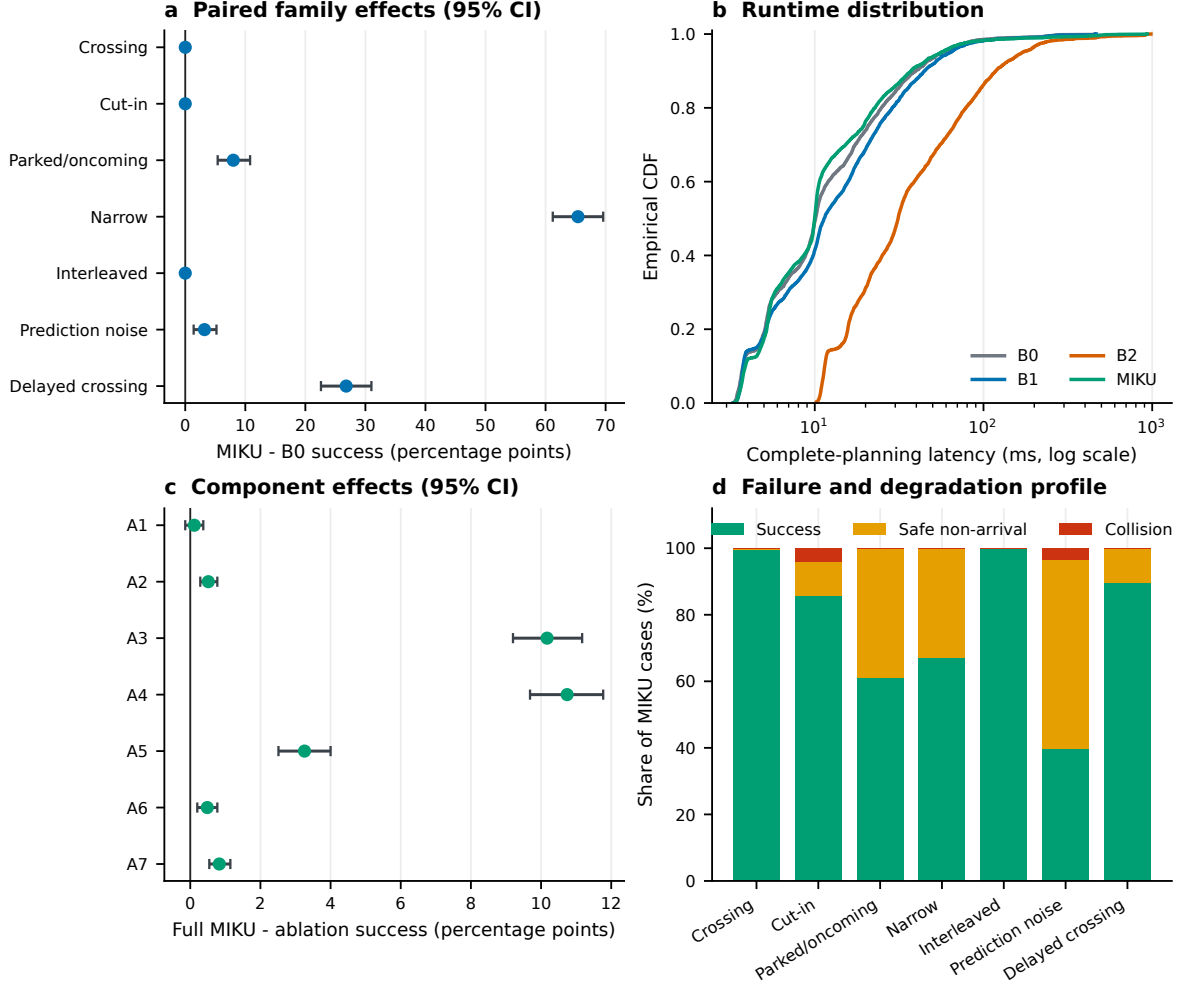


Figure 4. Evidence summary from the frozen 3,500-case protocol. Panel (a) shows paired MIKU–B0 success effects by scenario family with 95% bootstrap intervals; (b) shows complete-planning latency distributions; (c) shows full-method minus ablation success effects; and (d) decomposes MIKU outcomes into successful arrival, safe non-arrival, and collision.

percentage points, 95% CI $[-11.43, 5.71]$, $p = .754$). Median runtime was 1,835.43 ms for B3 and 10.44 ms for MIKU. Thus MIKU approaches the finite reference’s success rate at a much lower measured cost in this small, coarse grid experiment, while the two solvers optimize different representations.

6.2. RQ2: Mechanisms and ablations

The subtractive ablations isolate mechanisms on the same 3,500 scenarios. Removing Top- K spatial homotopy reduced success by 10.17 percentage points and removing the threat-aware margin reduced it by 10.74 points relative to full MIKU. Removing the temporal homotopy graph reduced delayed-crossing success from 89.6% to 66.8%, a 22.8-point family-specific loss. Removing the robust occupancy tube increased prediction-noise collision from 3.6% to 11.0%. Arrival projection and longitudinal grouping produce smaller aggregate changes, indicating that their value is concentrated in particular configurations rather than uniform across all families.

The mechanism cases provide a geometric explanation for these effects. In the narrow-passage case, the obstacle-wise policy raises and lowers opposing path bounds until the center interval becomes empty, whereas the group-level partition retains one continuous band. In the delayed-crossing family, the temporal graph preserves the before- and after-obstacle alternatives and enforces their causal order. These examples locate the failure mechanism; they are not substitutes

Table 3. Subtractive ablations on the same 3,500 paired scenarios. Each row removes one MIKU component.

Variant	Removed component	Success (%)	Collision (%)	Progress (%)
A1	Arrival-time projection	77.34	1.23	86.73
A2	Longitudinal grouping	76.94	1.14	85.89
A3	Top- K spatial homotopy	67.29	1.14	79.53
A4	Threat-aware margin	66.71	1.14	78.85
A5	Temporal homotopy graph	74.20	1.17	84.29
A6	Robust occupancy tube	76.97	2.20	86.60
A7	Alternating refinement	76.63	1.03	85.92
MIKU	None	77.46	1.14	86.45

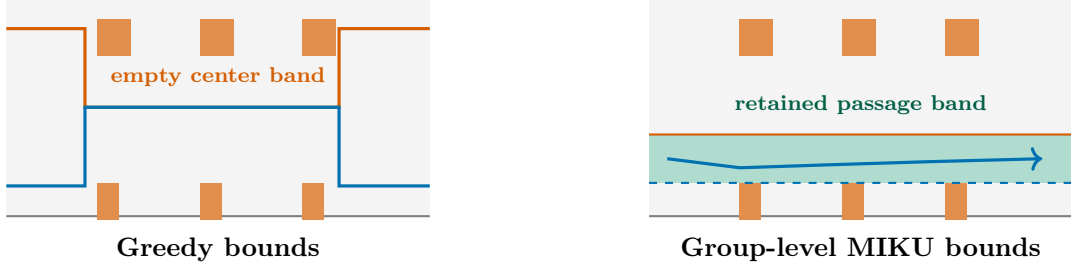


Figure 5. Illustration of the narrow multi-obstacle mechanism. Group-level partitioning retains a continuous center band after the obstacle-side decisions are coordinated.

for the family-level estimates.

6.3. RQ3: Robustness, side effects, and latency tails

The prediction-noise subgroup demonstrates both a benefit and a cost. MIKU reduces collisions by 7.0 percentage points relative to B0, but its mean progress is 59.73% versus 60.85% and its jerk RMS is 2.274 versus 2.234 m/s³. This is consistent with uncertainty-expanded occupancy being conservative in cases where the error bound is large relative to the available passage.

The complete-call runtime median and P95 are similar to or lower than B0, but the P99 tail is not. MIKU’s P99 is 221.80 ms, compared with 138.88 ms for B0; the empirical latency curve in Figure 4 shows the divergence in the upper tail. B2 is slower across all reported quantiles because it always performs three coordination iterations. These results support a typical-cycle efficiency statement, not a hard real-time guarantee.

The independent rolling-replanning protocol gives a second view of accumulated behavior. Across 700 cases, success increases from 62.29% to 72.29% (paired difference 10.0 percentage points, 95% CI 7.57–12.57), while collision remains 1.29% for both methods. Mean progress increases from 82.55% to 90.25%, but episode-runtime P95 increases from 827.20 to 1,361.51 ms. Because this protocol executes and replans rather than evaluating one open-loop call, it is reported as robustness evidence and not merged with Table 2.

6.4. RQ4: PVD software-interface evidence

The Apollo 11.0 implementation passes the generated path and ST constraints through the existing path-boundary and ST-boundary interfaces while retaining the downstream QP objectives and kinematic equalities. Three representative Dreamview cases show dynamic overtaking, narrow-passage traversal, and single-lane diversion. These cases demonstrate that the abstract constraint compilation has a compatible software insertion point and produces the intended qualitative behavior. They do not establish physical-road safety, native Apollo latency, hardware-in-the-loop performance, or complete end-to-end correctness.

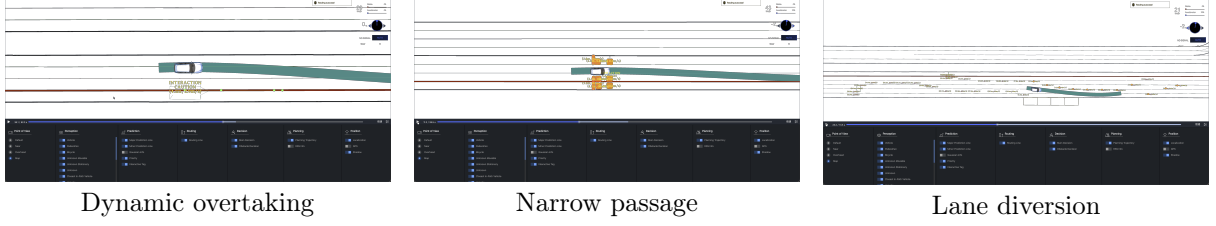


Figure 6. Representative Apollo Dreamview interface cases. The screenshots are qualitative integration evidence and are not used to compute the statistical results above.

7. Discussion

7.1. What the evidence establishes

The main result is a constraint-interface effect rather than a claim that one optimizer dominates all alternatives. MIKU retains the PVD solver but changes when and how obstacle information reaches it. The largest gains occur in narrow multi-obstacle and delayed-crossing configurations, precisely where a horizon union or an irreversible local side choice removes a feasible alternative. In configurations that are already easy for B0, such as the crossing-pedestrian and vehicle cut-in families, the aggregate success rate is unchanged. This pattern supports the proposed mechanism more directly than a uniform gain across all cases would.

The ablations provide a second level of support. Top- K spatial homotopy and threat-aware margins account for most of the overall success difference, while the temporal homotopy graph matters primarily when both before- and after-obstacle passage are reachable. The robust occupancy tube reduces collisions under prediction noise, but the corresponding progress and jerk costs show that safety-oriented expansion consumes usable space. These trade-offs are consistent with corridor and prediction-aware planning studies that treat feasibility, safety, and comfort as competing operational outcomes (F. Liu et al., 2024; B. Yang et al., 2023; Yoon et al., 2024).

7.2. Operational implication

For a PVD-based automated-vehicle stack, the practical design rule is to preserve interaction timing before the path boundary is finalized, then pass the selected temporal semantics to the speed stage. This can reduce unnecessary fallback stops without requiring a new nonlinear or mixed-integer downstream solver. The rule is most relevant to narrow work zones, temporary lane diversions, and mixed traffic in which several obstacles occupy a shared longitudinal component. It should be deployed with a latency monitor and a fallback policy because the upper runtime tail and uncertainty-expanded cases remain material.

7.3. Relation to alternatives

The finite-grid reference reaches a slightly higher success rate in the small comparison, but at a median measured runtime roughly two orders of magnitude larger. This comparison illustrates the quality-cost trade-off between an expanded joint search and a compiled PVD interface; it is not evidence that MIKU is globally better. Likewise, the iterative B2 baseline shows that simply repeating the PVD stages is insufficient when the missing information is a discarded lateral class. MIKU’s contribution is therefore the representation and transfer of constraints, not an assertion that iteration or joint search is unnecessary in every deployment.

7.4. Implications for evaluation

The separation between the 3,500-case numerical protocol, the 700-case rolling protocol, and Apollo interface cases is important for interpreting the results. The first estimates matched

scenario-level effects, the second tests accumulation under replanning, and the third checks an engineering insertion point. Combining them into one headline score would overstate external validity. A future evaluation should use public or field-collected trajectories, a matched recent external planner, and native runtime instrumentation under the target hardware budget.

8. Limitations

The evidence has five explicit boundaries. First, the main scenarios are generated on a straight road with constant-velocity obstacle models. This may overstate the regularity of the predicted occupancy and does not test curvature, lane topology, weather, or incident effects; the next study should replay public and field-collected trajectories across road geometries. Second, the prediction-noise model uses bounded position and velocity errors. Its collision result therefore applies to the stated error envelope, not to an arbitrary predictor; calibrated multi-modal prediction distributions are needed next. Third, B0–B2 are internal matched PVD variants and B3 is a finite coarse-grid reference. The paper makes no public SOTA or fair cross-dataset claim; a matched implementation of a recent external planner is the next comparison. Fourth, Apollo evidence covers interface compatibility and representative Dreamview behavior, not native Apollo performance, hardware-in-the-loop execution, physical-road testing, or an end-to-end safety proof. Native closed-loop instrumentation and controlled-road tests are required before deployment claims. Fifth, the maximum-gap theorem is exact for a fixed-section interval abstraction and the retained candidate set. It does not certify a continuous global optimum of the full nonconvex trajectory problem; a future formulation could study completeness under changing obstacle order.

9. Conclusion

This paper presented MIKU, an interaction-aware constraint compiler for path–velocity planning with dynamic multi-obstacle traffic. Arrival-time-dependent projection, continuous group-level partitioning, uncertainty-expanded occupancy, and temporal homotopy convert lost interaction information into SL and ST bounds while preserving the downstream path and speed QPs. The fixed-section theorem reduces side-assignment search to an $O(k \log k)$ scan, which matched exhaustive enumeration on 4,000 deterministic instances. In 3,500 paired scenarios, collision-free goal reaching increased from 62.7% to 77.5% and collision decreased from 2.2% to 1.1% relative to the time-blind baseline, with the largest effects in narrow and delayed-crossing configurations. Typical latency remained near 10 ms, but the P99 tail was higher and rolling replanning increased episode-runtime P95, so the result supports bounded simulation and interface claims rather than a hard real-time or road-deployment guarantee. The next study should validate the same constraint transfer with calibrated public or field trajectories and native runtime instrumentation.

Data availability statement

The anonymized replication package contains the frozen scenario generators, configuration files, aggregate results, raw rows, and analysis scripts. The package will be released in a persistent public repository upon acceptance. During review, the materials are available through the editorial process on reasonable request.

Code availability statement

The analysis and evaluation scripts needed to reproduce the reported numerical tables are included in the supplementary archive. The Apollo integration files and complete software package will

be released with the public repository after acceptance, subject to the licenses of the upstream Apollo components.

Funding

This work was supported by the Xiangyang Municipal Key Laboratory of Heterogeneous Big Data, the Hubei Superior and Distinctive Discipline Group of “New Energy Vehicle and Smart Transportation,” and the Key Research and Development Program of Hubei Province (Grant No. 2025BEB002).

Disclosure statement

The authors declare no competing interests.

Author contributions

JiaWang Liao: Conceptualization, methodology, software, formal analysis, data curation, visualization, and writing—original draft. YuFei Hu: Methodology, validation, investigation, and writing—review and editing. ChaoYang Shi: Software, validation, investigation, visualization, and writing—review and editing. ChengJiao Sun: Supervision, project administration, funding acquisition, conceptualization, and writing—review and editing. All authors approved the final manuscript.

AI use statement

OpenAI Codex (GPT-5; accessed September 2026) was used to translate, copyedit, and reorganize an existing author-prepared Chinese manuscript for journal presentation. It was not used to originate or conclude the scientific argument, or to generate the underlying data or analyses. Citation choices and bibliographic metadata were checked against the authors’ source materials and official records. JiaWang Liao reviewed and verified the complete English manuscript, including all scientific claims, equations, citations, and numerical results. The authors reviewed the applicable terms of use, confirm that this use is suitable for publication, and take full responsibility for the originality, integrity, and accuracy of the entire content, including the references.

References

- Chen, J., Zhan, W., & Tomizuka, M. (2019). Autonomous driving motion planning with constrained iterative LQR. *IEEE Transactions on Intelligent Vehicles*, 4(2), 244–254. doi: <https://doi.org/10.1109/TIV.2019.2904385>
- Fan, H., Zhu, F., Liu, C., Zhang, L., Zhuang, L., Li, D., ... Kong, Q. (2018). *Baidu Apollo EM motion planner*.
- González, D., Pérez, J., Milanés, V., & Nashashibi, F. (2016). A review of motion planning techniques for automated vehicles. *IEEE Transactions on Intelligent Transportation Systems*, 17(4), 1135–1145. doi: <https://doi.org/10.1109/TITS.2015.2498841>
- Han, W., Leng, B., Zhang, P., & Xiong, L. (2026). Safety-critical kinematically-executable overtake planning via contingency path-speed iterative algorithm for automated valet parking. *IET Intelligent Transport Systems*, 20(1), e70140. doi: <https://doi.org/10.1049/itr2.70140>

- Hu, J., Zhang, Z., Chen, R., Chen, R., Liu, H., Zhu, Q., & Chen, H. (2023). Spatiotemporal joint planning for intelligent vehicles based on an improved hybrid A*. *Automotive Engineering*. (In Chinese; title translated for this manuscript) doi: <https://doi.org/10.19562/j.chinasae.qcgc.2023.07.003>
- Hu, J., Zheng, J., Zhou, S., Zhao, W., Zhang, Z., & Yao, M. (2025). Spatiotemporal joint trajectory planning for intelligent vehicles on structured roads. *Automotive Engineering*. (In Chinese; title translated for this manuscript) doi: <https://doi.org/10.19562/j.chinasae.qcgc.2025.05.003>
- Jeong, Y., & Yi, K. (2021). Target vehicle motion prediction-based motion planning framework for autonomous driving in uncontrolled intersections. *IEEE Transactions on Intelligent Transportation Systems*, 22(1), 168–177. doi: <https://doi.org/10.1109/TITS.2019.2955721>
- Kant, K., & Zucker, S. W. (1986). Toward efficient trajectory planning: The path-velocity decomposition. *The International Journal of Robotics Research*, 5(3), 72–89. doi: <https://doi.org/10.1177/027836498600500304>
- Kessler, T., Esterle, K., & Knoll, A. (2023). Mixed-integer motion planning on German roads within the Apollo driving stack. *IEEE Transactions on Intelligent Vehicles*, 8(1), 851–867. doi: <https://doi.org/10.1109/TIV.2022.3162671>
- Liu, C., Zhan, W., & Tomizuka, M. (2017). Speed profile planning in dynamic environments via temporal optimization. In *2017 IEEE Intelligent Vehicles Symposium* (pp. 154–159). doi: <https://doi.org/10.1109/IVS.2017.7995713>
- Liu, F., Zhao, X., Su, W., & Wen, Y. (2024). Dynamic path-speed planning algorithm for autonomous driving on structured roads. *Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering*, 238(10-11), 3172–3193. doi: <https://doi.org/10.1177/09544070231181626>
- Liu, S., Watterson, M., Mohta, K., Sun, K., Bhattacharya, S., Taylor, C. J., & Kumar, V. (2017). Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-D complex environments. *IEEE Robotics and Automation Letters*, 2(3), 1688–1695. doi: <https://doi.org/10.1109/LRA.2017.2663526>
- McNaughton, M., Urmson, C., Dolan, J. M., & Lee, J.-W. (2011). Motion planning for autonomous driving with a conformal spatiotemporal lattice. In *2011 IEEE International Conference on Robotics and Automation* (pp. 4889–4895). doi: <https://doi.org/10.1109/ICRA.2011.5980223>
- Minderhoud, M. M., & Bovy, P. H. (2001). Extended time-to-collision measures for road traffic safety assessment. *Accident Analysis & Prevention*, 33(1), 89–97. doi: [https://doi.org/10.1016/S0001-4575\(00\)00019-1](https://doi.org/10.1016/S0001-4575(00)00019-1)
- Paden, B., Čáp, M., Yong, S. Z., Yershov, D., & Frazzoli, E. (2016). A survey of motion planning and control techniques for self-driving urban vehicles. *IEEE Transactions on Intelligent Vehicles*, 1(1), 33–55. doi: <https://doi.org/10.1109/TIV.2016.2578706>
- Qiao, Y. (2025). *Study on a spatiotemporal joint trajectory-planning algorithm for autonomous driving on structured roads* (Master’s thesis, Harbin Institute of Technology). (In Chinese; title translated for this manuscript) doi: <https://doi.org/10.27061/d.cnki.ghgdu.2025.003637>
- Schwarting, W., Alonso-Mora, J., & Rus, D. (2018). Planning and decision-making for autonomous vehicles. *Annual Review of Control, Robotics, and Autonomous Systems*, 1(1), 187–210. doi: <https://doi.org/10.1146/annurev-control-060117-105157>
- Tariq, F. M., Yeh, Z.-H., Singh, A., Isele, D., & Bae, S. (2025). Frenet corridor planner: An optimal local path planning framework for autonomous driving. In *2025 IEEE Intelligent Vehicles Symposium (IV)* (pp. 686–693). doi: <https://doi.org/10.1109/IV64158.2025.11097649>
- Wang, H., Zhang, L., Kong, Q., Zhu, W., Zheng, J., Zhuang, L., & Xu, X. (2022). Motion planning in complex urban environments: An industrial application on autonomous last-mile delivery vehicles. *Journal of Field Robotics*, 39(8), 1258–1285. doi: <https://doi.org/10.1002/rob.22107>
- Werling, M., Ziegler, J., Kammel, S., & Thrun, S. (2010). Optimal trajectory generation for dynamic street scenarios in a Frenet frame. In *2010 IEEE International Conference on Robotics and Automation* (pp. 987–993). doi: <https://doi.org/10.1109/ROBOT.2010.5509799>
- Yang, B., Yan, S., Wang, Z., & Nakano, K. (2023). Prediction based trajectory planning for safe interactions between autonomous vehicles and moving pedestrians in shared spaces. *IEEE Transactions on Intelligent Transportation Systems*, 24(10), 10513–10524. doi: <https://doi.org/10.1109/TITS.2023.3281157>
- Yang, X., & Li, H. (2021). *IA planner: Motion planning using instantaneous analysis for autonomous vehicle in the dense dynamic scenarios on highways*. Retrieved from <https://arxiv.org/abs/2103.10909>
- Yoon, Y., Kim, C., Lee, H., Seo, D., & Yi, K. (2024). Spatio-temporal corridor-based motion planning of lane change maneuver for autonomous driving in multi-vehicle traffic. *IEEE Transactions on Intelligent Transportation Systems*, 25(10), 13163–13183. doi: <https://doi.org/10.1109/TITS.2024.3388380>

- Zhang, Y., Sun, H., Zhou, J., Pan, J., Hu, J., & Miao, J. (2020). Optimal vehicle path planning using quadratic optimization for Baidu Apollo open platform. In *2020 IEEE Intelligent Vehicles Symposium (IV)* (pp. 978–984). doi: <https://doi.org/10.1109/IV47402.2020.9304787>
- Zhang, Z., Wang, C., Zhao, W., Cao, M., Liu, J., & Xu, K. (2023). Path-speed decoupling planning method based on risk cooperative game for intelligent vehicles. *IEEE Transactions on Transportation Electrification*. doi: <https://doi.org/10.1109/TTE.2023.3316124>
- Zhou, J., He, R., Wang, Y., Jiang, S., Zhu, Z., Hu, J., ... Luo, Q. (2021). Autonomous driving trajectory optimization with dual-loop iterative anchoring path smoothing and piecewise-jerk speed optimization. *IEEE Robotics and Automation Letters*, 6(2), 439–446. doi: <https://doi.org/10.1109/LRA.2020.3045925>